

L'architecture de la base de données a été conçue pour refléter fidèlement les besoins de gestion financière multi-utilisateurs et multi-devises identifiés lors de l'analyse.

Approche de Modélisation

Le système utilise une base de données relationnelle MySQL, structurée pour assurer une forte cohérence des données. L'entité **Ménage** agit comme le pivot central de l'architecture.

Détail technique des modules

➤ Gestion des Identités et Accès :

- ✓ La table Utilisateurs gère l'authentification (email unique, mot de passe hashé).
- ✓ Une table de liaison Membres_Menage a été implémentée pour gérer la relation **plusieurs-à-plusieurs** : un utilisateur peut être membre de plusieurs ménages, et un ménage regroupe plusieurs utilisateurs.

➤ Logique de Flux Financiers (Transactions) :

- ✓ Plutôt que de séparer les revenus et les dépenses dans deux tables distinctes, nous avons opté pour une approche de **classe parente/abstraite** représentée par la table Transactions.
- ✓ L'utilisation d'un ENUM('Revenu', 'Depense') permet de simplifier les calculs de solde via des requêtes SUM() avec condition, optimisant ainsi les performances du moteur SQL.

➤ Contraintes d'Intégrité et Business Logic :

- ✓ **Comptes & Devises** : Chaque compte est lié à une devise spécifique et appartient à un ménage. Cela permet de gérer des portefeuilles en **USD** et en **CDF** simultanément.
- ✓ **Budgets et Alertes** : La table Budgets définit des plafonds financiers. La table Alertes y est rattachée par une clé étrangère, permettant au Back-End de déclencher des notifications basées sur le dépassement du montant limite_montant.

➤ Gestion du Patrimoine :

Les dettes et créances sont centralisées dans une table Patrimoine différenciée par le champ `type_engagement`, permettant de suivre les montants dus à des tiers ou par des tiers.

Team-3